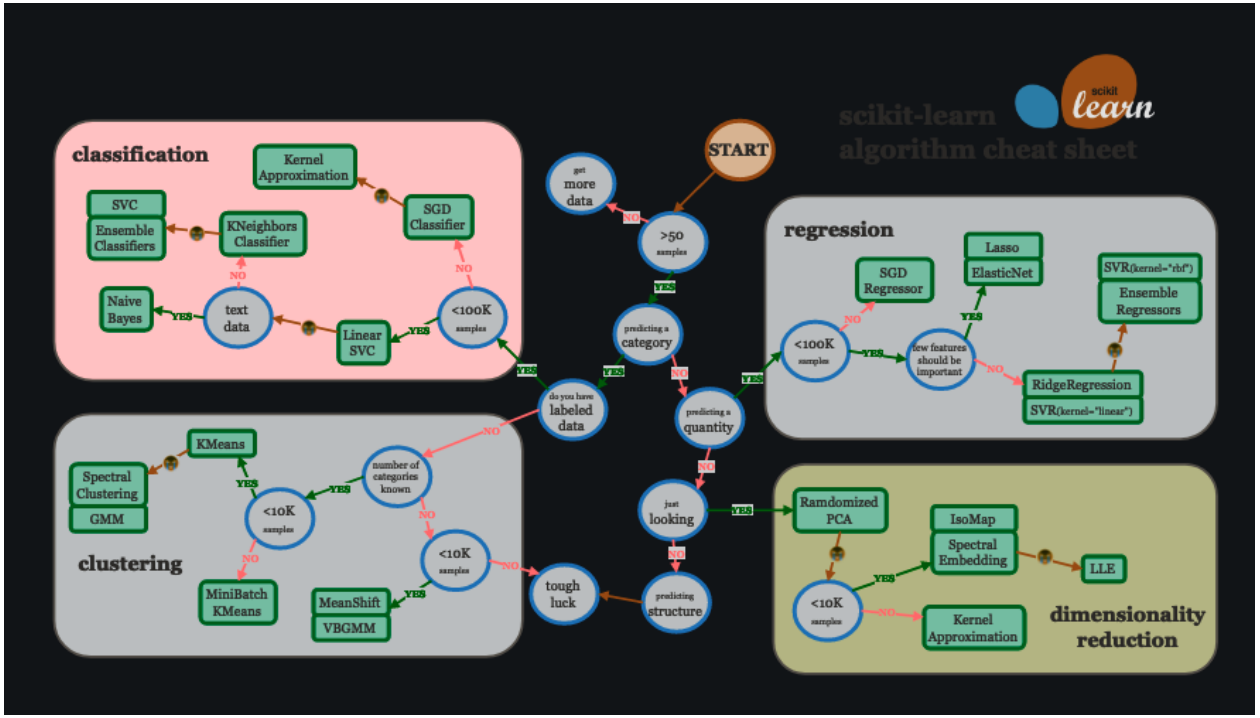
Fondamentaux théoriques du machine learning



Overview of lecture 8

Adaptivity

- No free lunch theorems
- Adaptivity

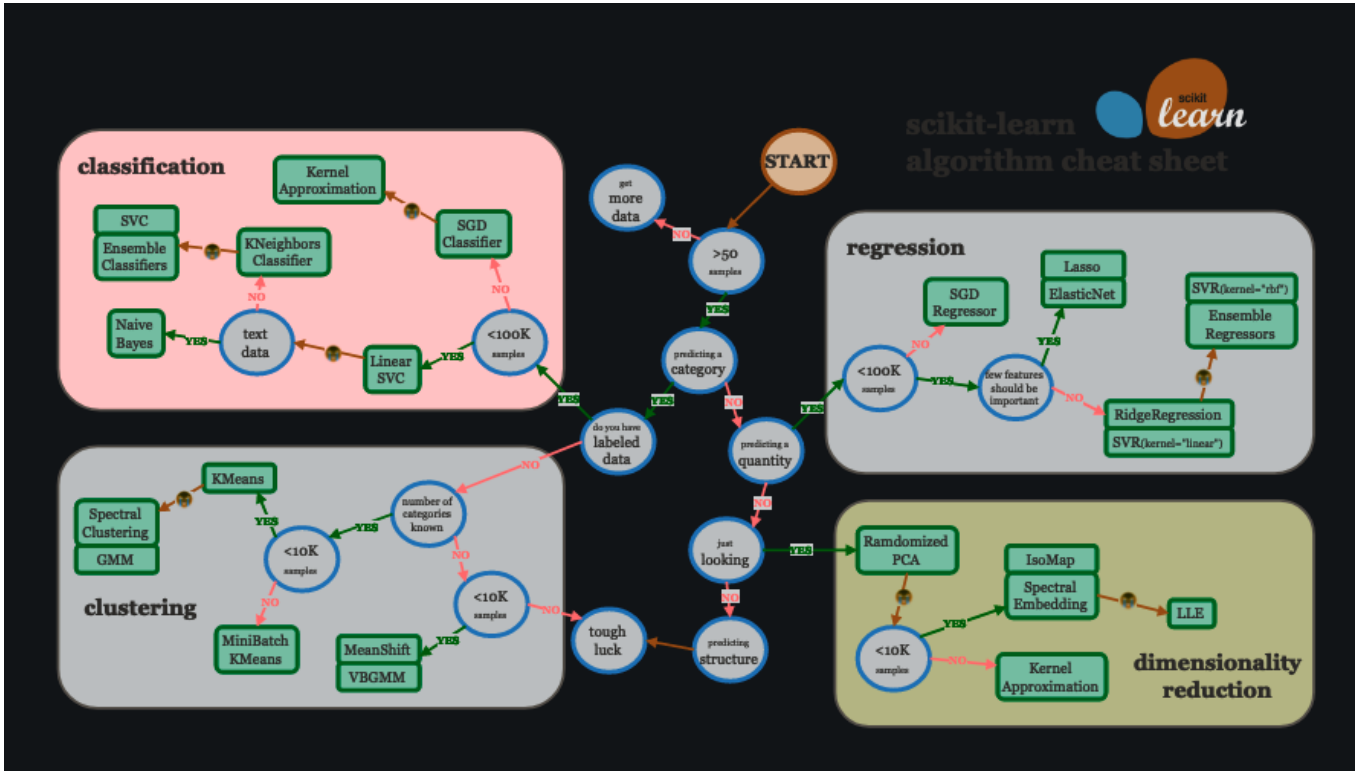
Reinforcement Learning

- Dynamic programming
- Value Iteration

Scoring, multiclass problems, cross validation

- R2 for Regression
- Binary classification
- Multi-class classification
- Classification threshold

ML map



https://scikit-learn.org/stable/tutorial/machine_learning_map/

No free lunch theorems

$\mathcal{A}$: learning rule. Takes the dataset D_n as input and outputs an estimator f_n (for instance based on empirical risk minimization, local averaging, etc).
There are several no free lunch theorems.

No free lunch theorems

Theorem

No free lunch - fixed n
We consider a binary classification task with "0-1"-loss, and $\mathcal{X}$ infinite.
We note $\mathcal{P}$ the set of all probability distributions on $\mathcal{X} \times \{0, 1\}$.
For any $n > 0$ and any learning rule $\mathcal{A}$

$$\sup_{dp \in \mathcal{P}} E\Big[R_{dp}\big(\mathcal{A}(D_n(dp))\big)\Big] - R^*_{dp} \geq \frac{1}{2} \tag{1}$$

We write $D_n(dp)$ in order to emphasize that the dataset is sampled randomly from the distribution dp, with n samples.

No free lunch

- For any learning rule, there exists a distribution for which this learning rule performs badly.
- No method is universal and can have a good convergence rate on all problems.

However, considering **all** problems is probably not relevant for machine learning.

Adaptivity

If the learning rule improves (faster convergence rate) when we add a property on the problem (for instance, regularity of the target function), we say that we have **adaptivity** to this property.
For instance : gradient descent is adaptive to the strong convexity of the target function, since with a proper choice of the learning rate γ, the convergence rate is exponential, at a rate that is larger when the strong convexity constant μ is larger.
There are several forms of adaptivity.

Most general case

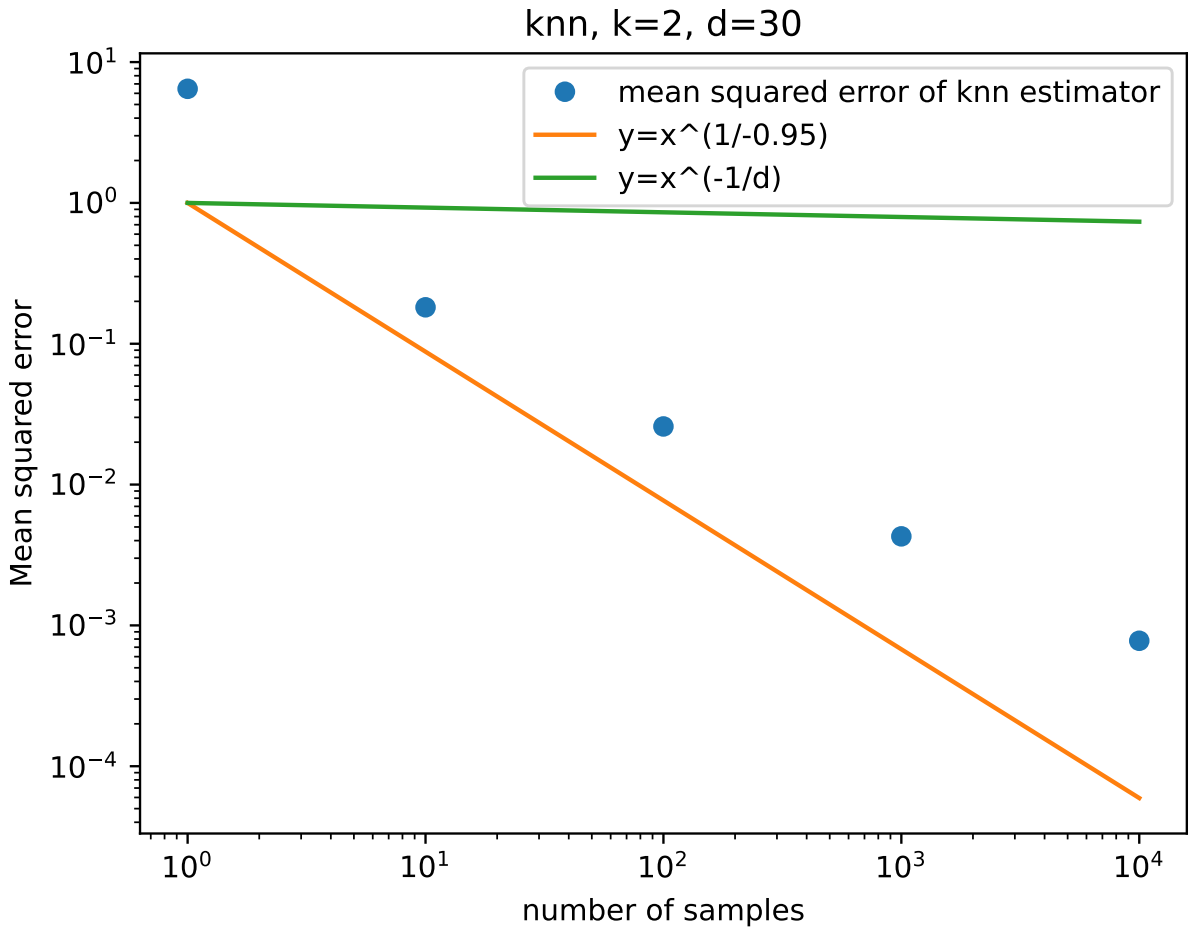
The target is just Lipshitz-continuous, no extra-hypothesis. In this case the optimal rate is of the form $\mathcal{O}(n^{-\frac{2}{d+2}})$ (curse of dimensionality) for all learning rules.
To reach an error of $\epsilon < 1$, we need a number of samples that is $\mathcal{O}((\frac{1}{\epsilon})^{\frac{d}{2}+1})$, thus exponential in d.

Adaptivity to the input space

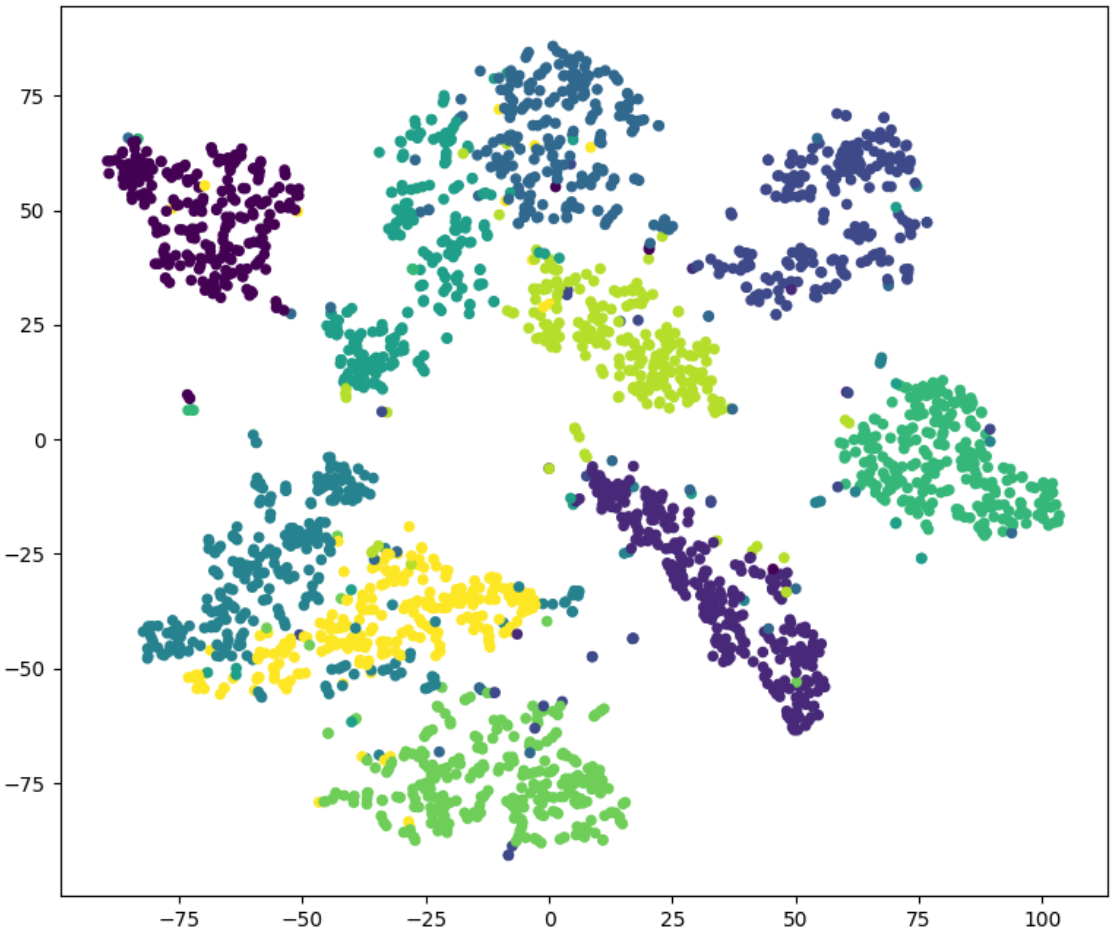
If the input data lie on a submanifold (e.g. a subspace) of $\mathbb{R}^d$ of lower dimension than d, most methods adapt to this property.

Adaptivity to a lower dimensional support

We saw an example of this behavior during a practical seccion.



Lower dimensional manifolds



Adaptivity to the regularity of the target function

If the target is smoother (meaning that all derivatives up to order m are bounded), kernel methods (here, positive-definite kernels) and neural network adapt, if well optimized and regularized.
The convergence rate can become $\mathcal{O}(n^{-\frac{m}{d}})$.

Adaptivity to latent variables

If the target function depends only on a k dimensional linear projection of the data, neural networks adapt, if well optimized. The rate can become $O(n^{-\frac{m}{k}})$.

`https://francisbach.com/quest-for-adaptivity/`

Sutton textbook

http://incompleteideas.net/book/the-book-2nd.html

- RL has many applications and **Deep Reinforcement Learning** has received a lot of attention for more than 15 years now.
- Sutton textbook : `http://incompleteideas.net/book/the-book-2nd.html`

- Atari games

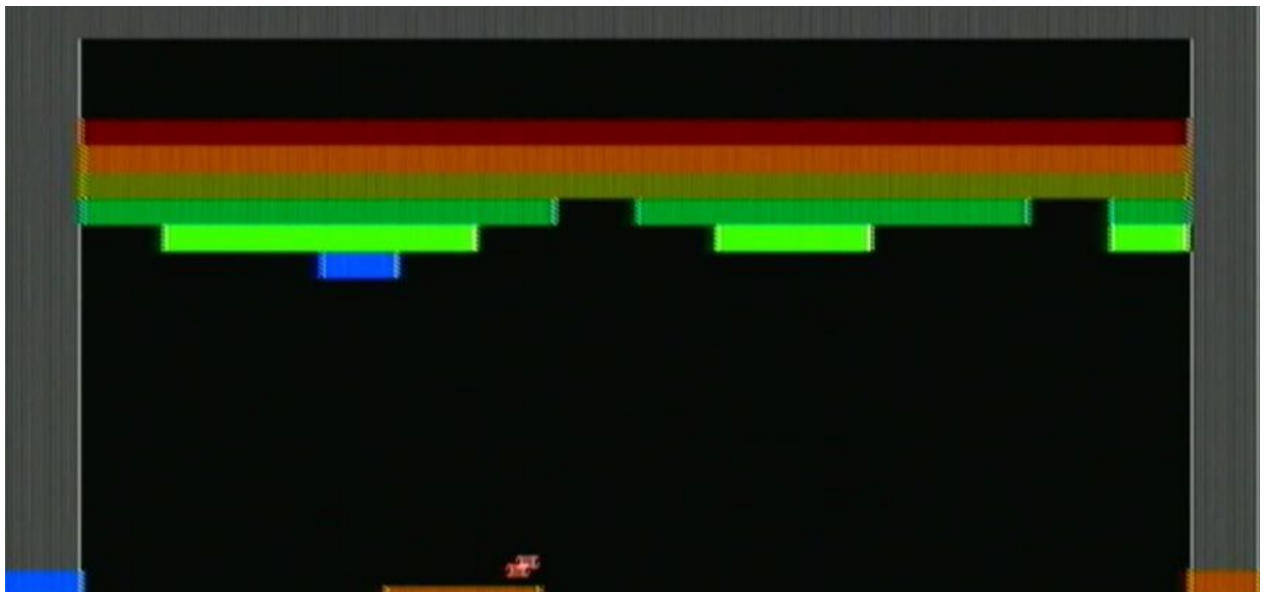


Figure – [Mnih et al., 2013]

- AlphaGo

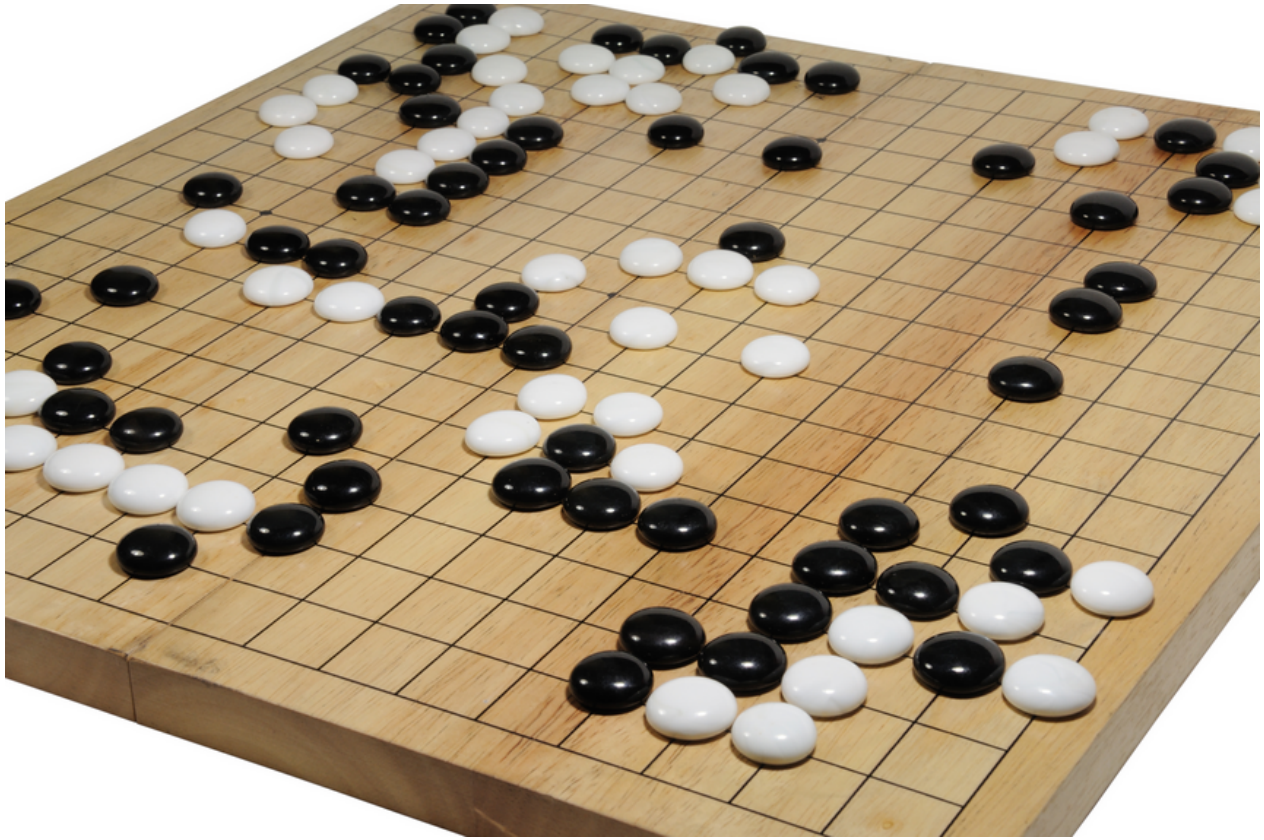


Figure – Go game, beaten by AlphaGo in 2017 [Silver et al., 2016]

- Reinforcement Learning is also being used in the community of **Computationnal neuroscience.**

Supervised learning and Correction

- In **supervised learning**, the supervisor indicates the **expected answer** the model should answer.
- The feedback does not depend on the action performed by the model (for instance the prediction from the model)
- We say that the model receives an **instructive feedback**.
- The model must then **update itself** based on this answer.

Cost sensitive learning

- In **Cost sensitive learning**, the situation is different.
- The agent receives an **evaluative feedback**. The feedack depends on the action performed by the agent.
- **Examples :**
  - AI playing a game and receiving "victory" or "defeat" as a feedback.
  - Child playing with toys.

Reinforcement learning

- **Reinforcement learning** is a particular case of cost-sensitive learning.
- In reinforcement learning, the feedback is a **real number**.
- **Example :** amount of coins won after a poker turn.

Reinforcement learning

- First, the agent does not know if a reward is good or bad per se.
- A reward of -10 good be good or bad depending on the other rewards that are possible to obtain.
- Most of the time, the objective of the agent will be to optimize the **agregation of rewards**.

Reinforcement learning

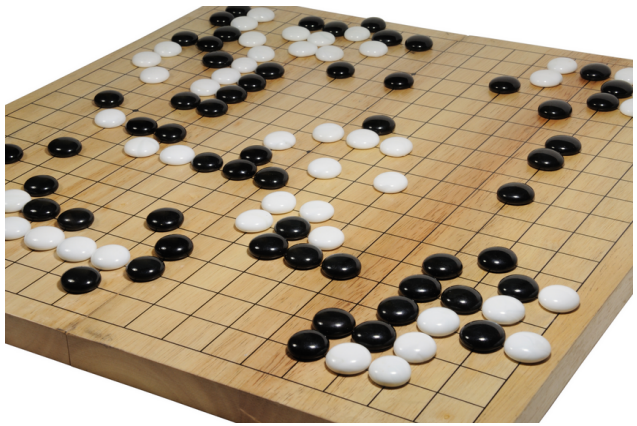
- The agent lives in a world E, and can be in several states s. The agent performs **actions** a and receives rewards r.
- **Examples :**
  - world $= \mathbb{R}^2$
  - state $=$ position
  - actions $=$ moving somewhere
  - reward $=$ amount of food found

Formalization

- There are many aspects of the problem that we need to formalize. Several formalizations are possible depending on the situation.
- We will consider **discrete spaces** :
  - the time will be discrete
  - the number of possible states will be **finite**
  - the number of possible actions will be **finite**
- Continuous spaces are also available for RL. In those cases the objects are slightly different, and the optimization procedures also differ. For an introductory course, discrete spaces are more suitable.

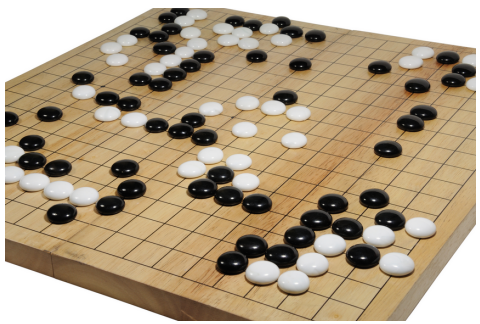
Question

- We will consider **discrete spaces** :
  - the time will be discrete
  - the number of possible states will be **finite**
  - the number of possible actions will be **finite**
- Are these hypotheses valid in the case of AlphaGo ?



Question

- We will consider **discrete spaces** :
  - the time will be discrete
  - the number of possible states will be **finite**
  - the number of possible actions will be **finite**
- Are these hypothesis valid in the case of AlphaGo ?



- Yes ! This shows that discrete spaces can still describe very complex problems.

Formalization

- we will write :
  - S_t : state at time t
  - R_t : reward received at time t
  - A_t : action performed at time t
- the actions are chosen according to a **policy** π

Action - reward loop

54 CHAPTER 3. FINITE MARKOV DECISION PROCESSES

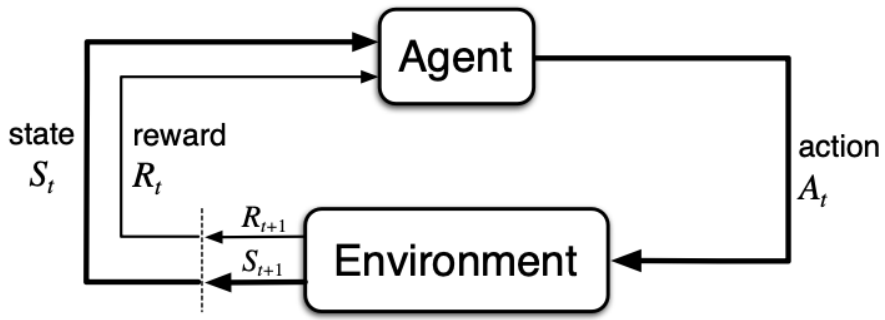


Figure 3.1: The agent–environment interaction in reinforcement learning.

Figure – Image taken from the Sutton book, page 68.

http://incompleteideas.net/book/the-book-2nd.html

Policies

The policy is a map that determines the action chosen by the agent. π is a function of the current state.

- It can be **deterministic :** the action chosen is chosen with probability 1.
- Or **stochastic :** the action performed in a given state is drawn from a **distribution**.

Two levels of randomness

- The policy can be deterministic or stochastic.
- But the result of an action could also be stochastic ! This is called a **stochastic transition function**.

Two levels of randomness

- The policy can be deterministic or stochastic.
- But the result of an action could also be stochastic ! This is called a **stochastic transition function**.

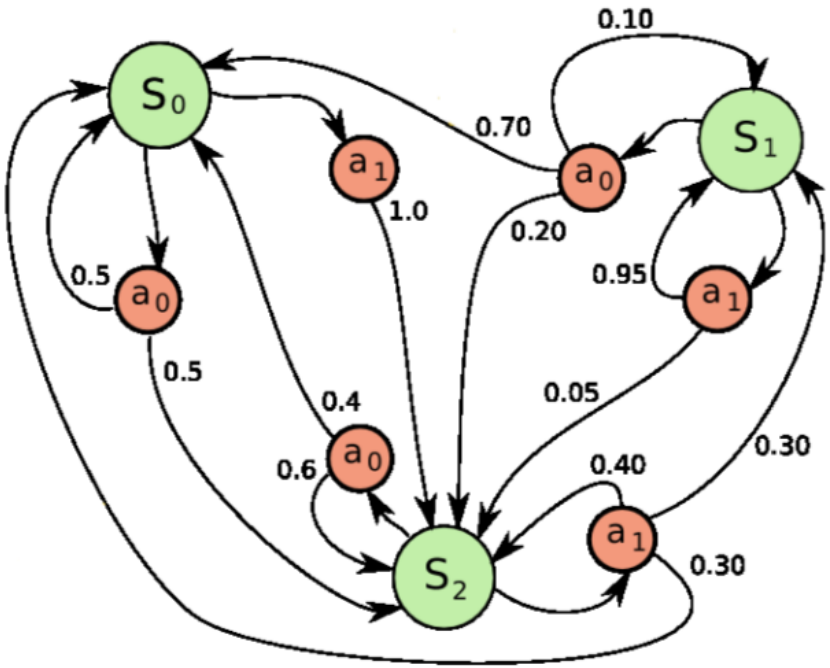


Figure – A stochastic policy with a stochastic transition function.

Exercice 1 :

- What is the probability of staying in state S_0 when performing an action from S_0 ? and from S_1 and S_2 ?

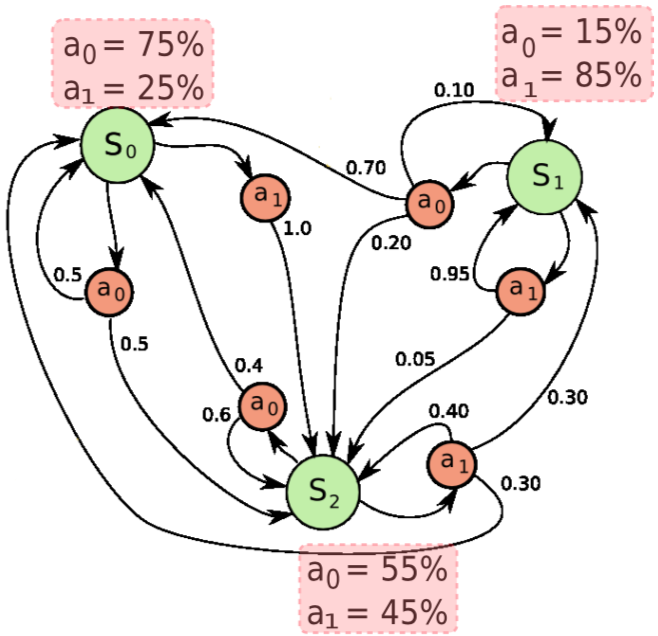


Figure – A stochastic policy with a stochastic transition function.

Agregation of rewards

- The agent wants to optimize the **agregation of the rewards.**
- There are several ways to agregate the rewards.

Returns

We introduce the **return** G_t.

- Episodic case (finite number N of steps) :

$$G_t = R_{t+1} + R_{t+2} + \cdots + R_{t+N} \tag{2}$$

- Continuing tasks :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \tag{3}$$

$\gamma \in [0, 1[$ is the **discount factor**

Value function

Given a fixed policy π, the value function $v_\pi(s)$ quantifies how good a state s is.

$$v_\pi(s) = E[G_t | S_t = s] \tag{4}$$

(the expectation is taken over the next actions, following the policy π).

- G_t : return obtained after the word is in state s at time t.
- E : expected value

The value function is unknown first, this is what we want to learn !

The Bellman equation

Given a fixed policy π, and a state s, we can write a recursive relationship between $v_\pi(s)$ and the values v_π of the next possible successor states (see the Sutton book for the general form of the equation 3.12 page 85).

Markov hypothesis

A assumption necessary to enable computations, but not always exactly true.
`https://en.wikipedia.org/wiki/Markov_property`

ϵ-greedy policy

A way to tackle the **exploitation-exploration compromise**.

- with probability $1 - \epsilon$: go to the best known reward (exploitation).
- with probability ϵ : perform a random action (exploration).

"RL is a science, but dealing with the exploration-exploitation compromise is an art" (Sutton)

World

We will illustrate reinforcement learning with a specific case (deterministic transition function in a "small" world)

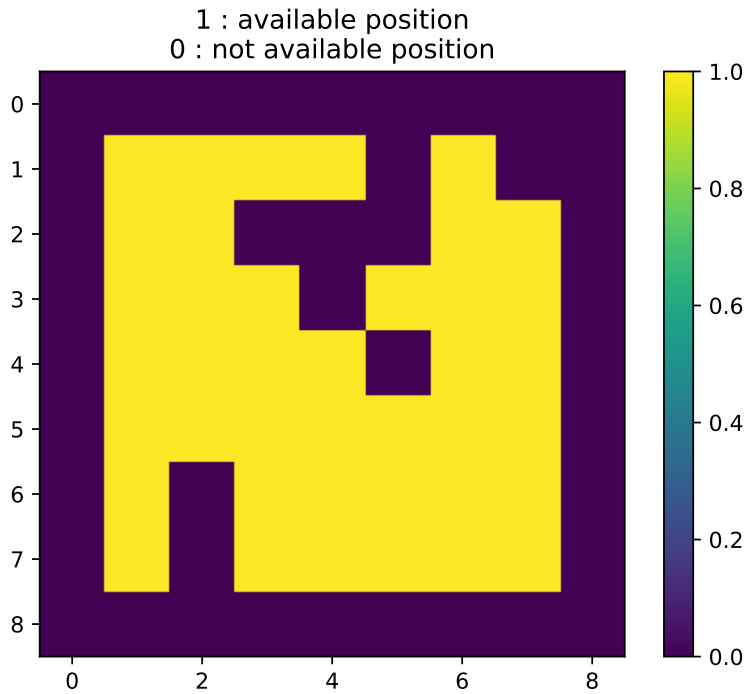


Figure – 2 dimensional world.

Reward

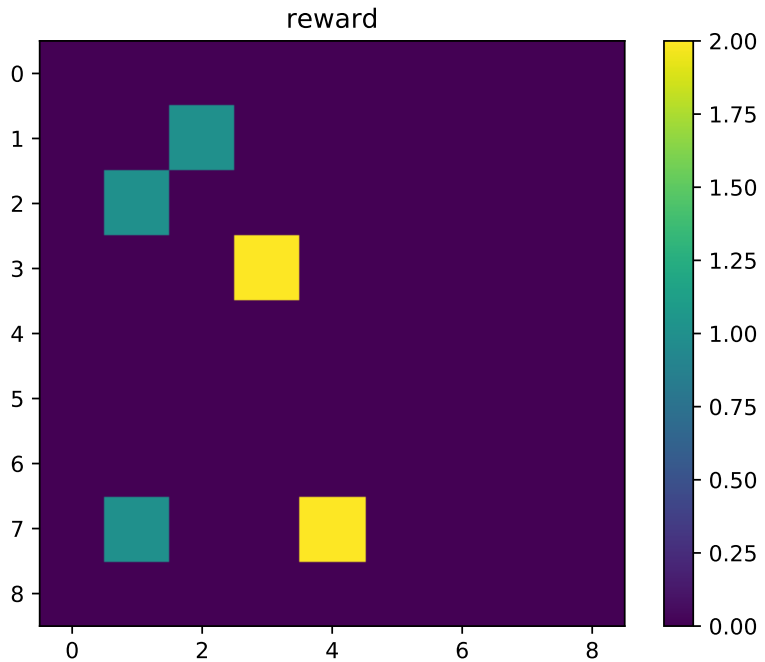


Figure – Reward function.

2D world

- Our agent can move to the 8 neighbor positions (if the positions are available)

Optimal value functions

We look for the value of the **optimal policy** π^*, defined by the fact that it has the best value function among all policies.

Bellmann optimality equation

$$
\begin{aligned}
v_*(s) &= \max_{a \in \text{available actions}} E[G_t | S_t = s, A_t = a] \\
&= \max_{a \in \text{available actions}} E[\sum_{k \geq 0} \gamma^k R_{t+k+1} | S_t = s, A_t = a] \\
&= \max_{a \in \text{available actions}} E[R_{t+1} + \gamma \sum_{k \geq 1} \gamma^{k-1} R_{t+k+1} | S_t = s, A_t = a] \\
&= \max_{a \in \text{available actions}} E[R_{t+1} + \gamma \sum_{k \geq 0} \gamma^k R_{t+k+2} | S_t = s, A_t = a] \\
&= \max_{a \in \text{available actions}} E[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a]
\end{aligned}
\tag{5}
$$

Bellman optimality equation

In the case of our simple 2D deterministic world, the Bellman optimality equation 5 takes a simpler form !

$$v_*(s) = \max_{a \in \text{available actions}} R_{t+1} + \gamma v_*(S_{t+1}) \tag{6}$$

The expected value is replaced by a deterministic value, because in this specific case :

- S_{t+1} is deterministic, given S_t and A_t.
- The reward is deterministic.

Value Iteration

- Value iteration belongs to dynamic programming methods. They are a specific case of RL where a perfect model of the environment is assumed.
- In value iteration, equation 5 is used as an **update rule** at each time step.
- First, the initial Value function for all the states is 0.
- Then we propagate the information about the rewards between the states, in order to **update the value function** (sweep)

$$\forall s \in V(s) \leftarrow \max_a \left(R_{t+1} + \gamma V(s_{t+1}) | S_t = s, A_t = a\right) \quad (7)$$

- In parallel, we explore the world to learn about the distribution of rewards.

Value iteration

Initially, no reward is known, no value is attributed to any state.

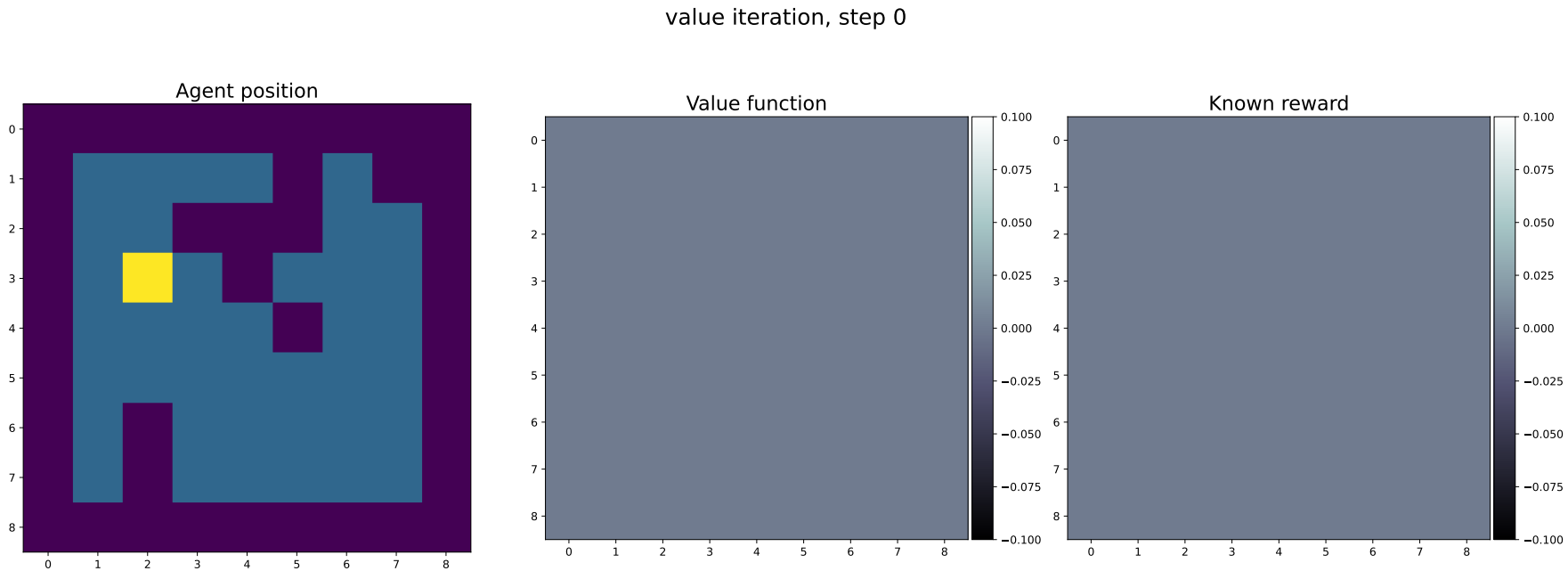


Figure – Iteration 0

Value iteration

At step 3 a reward is found, information propagates.

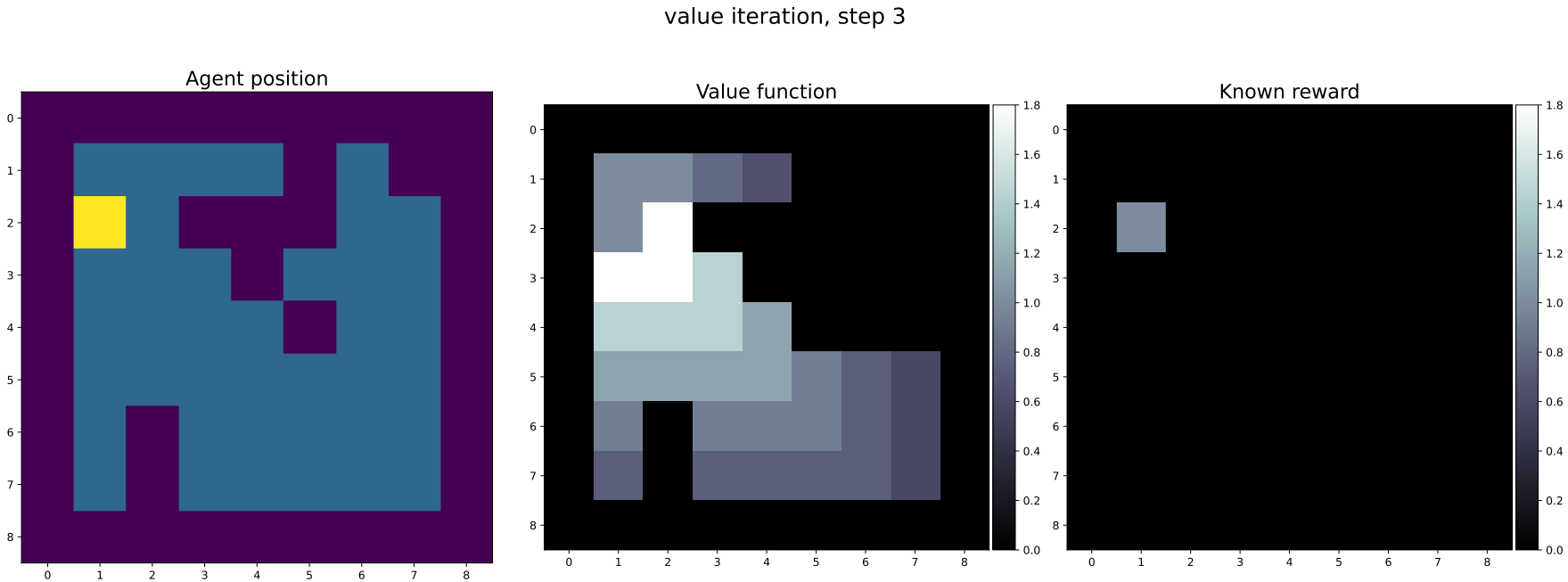


Figure – Iteration 3

Value iteration

Information propagation.

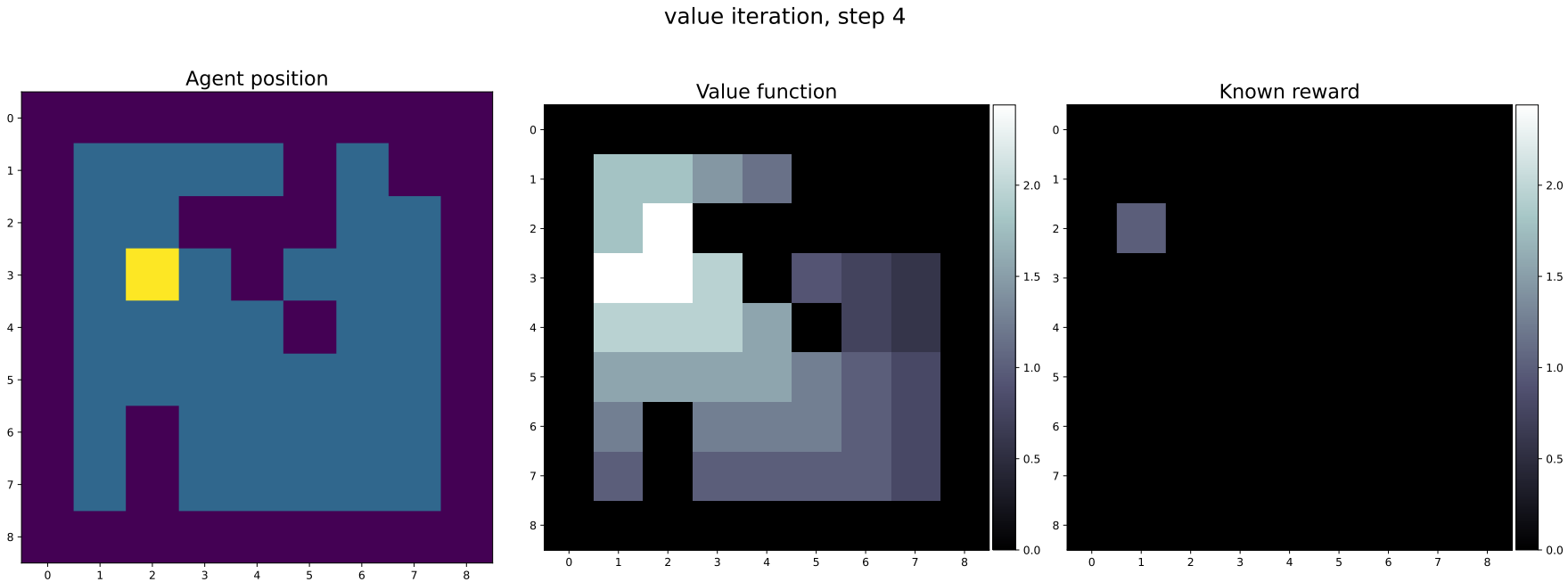


Figure – Iteration 4

Value iteration

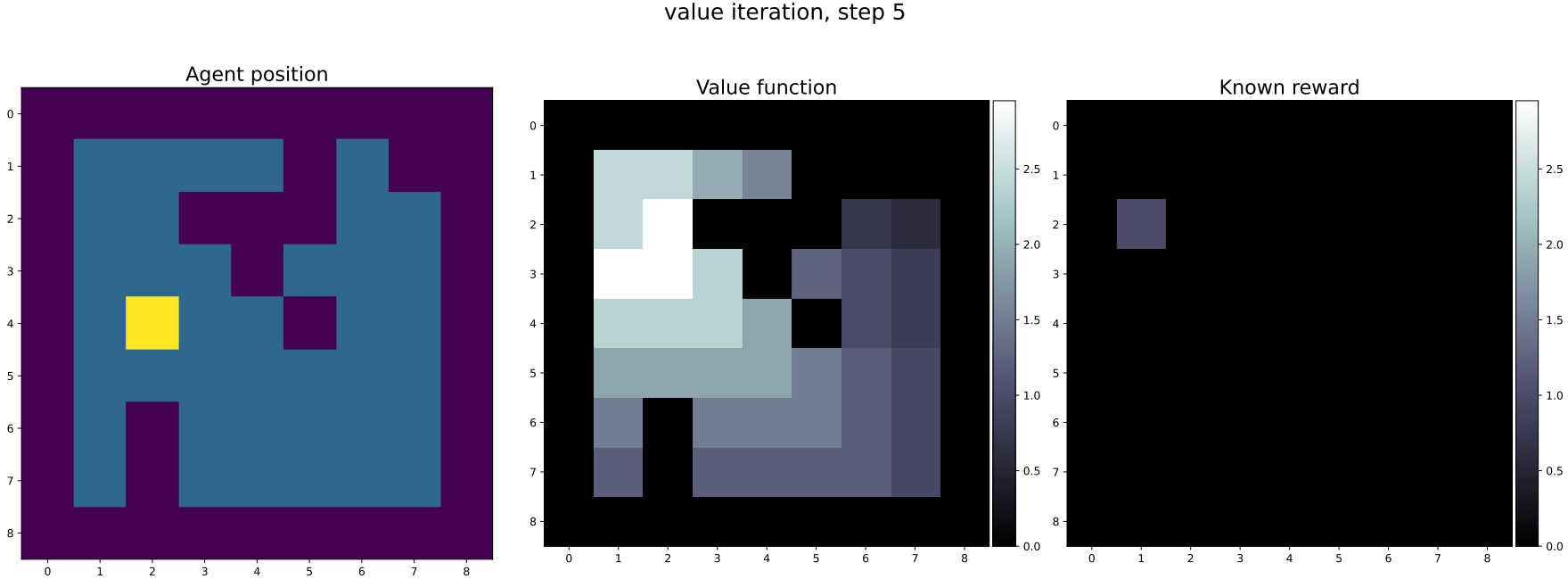


Figure – Iteration 5

Value iteration

Another reward is found at step 7.

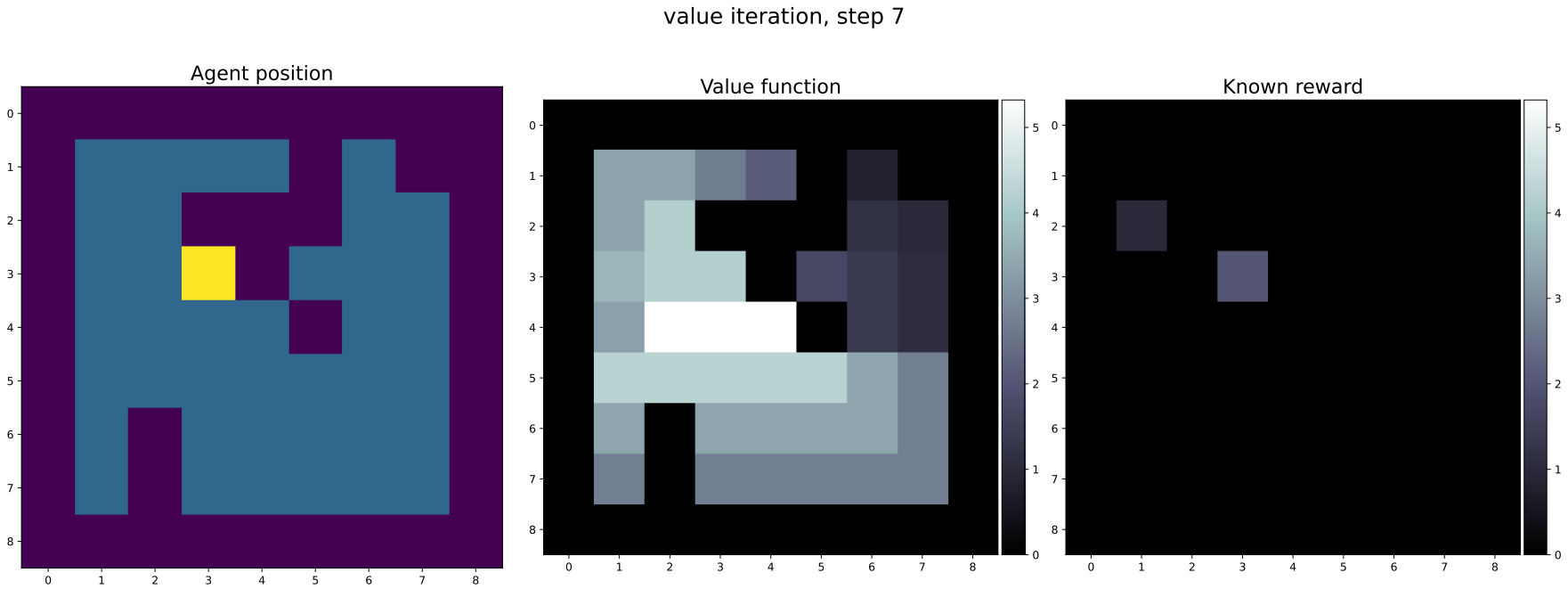


Figure – Iteration 7

Value iteration

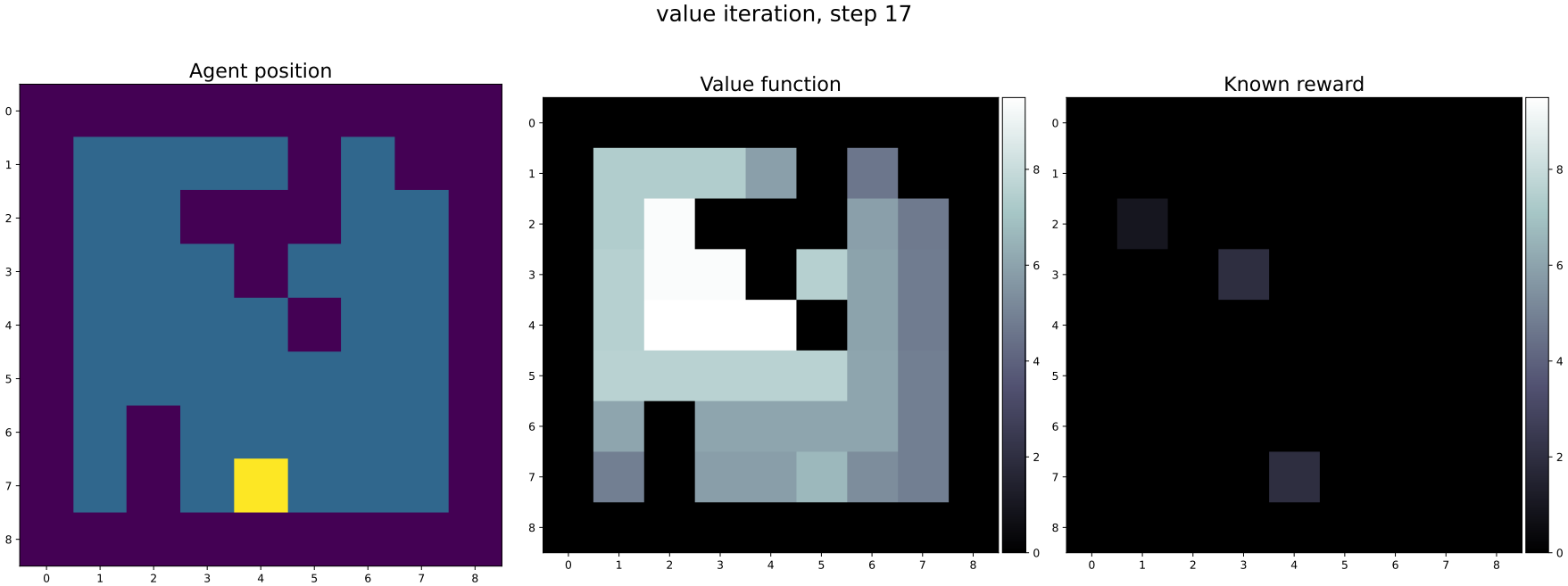


Figure – Iteration 17

Value iteration

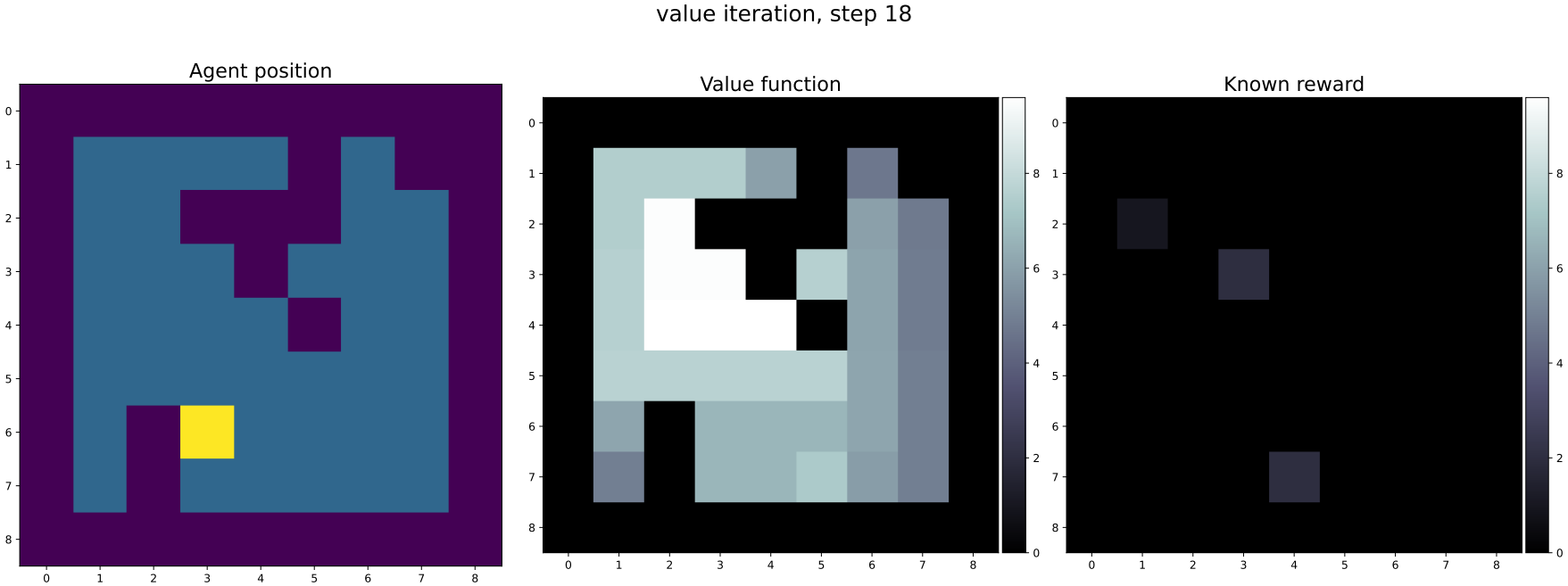


Figure – Iteration 18

Value iteration

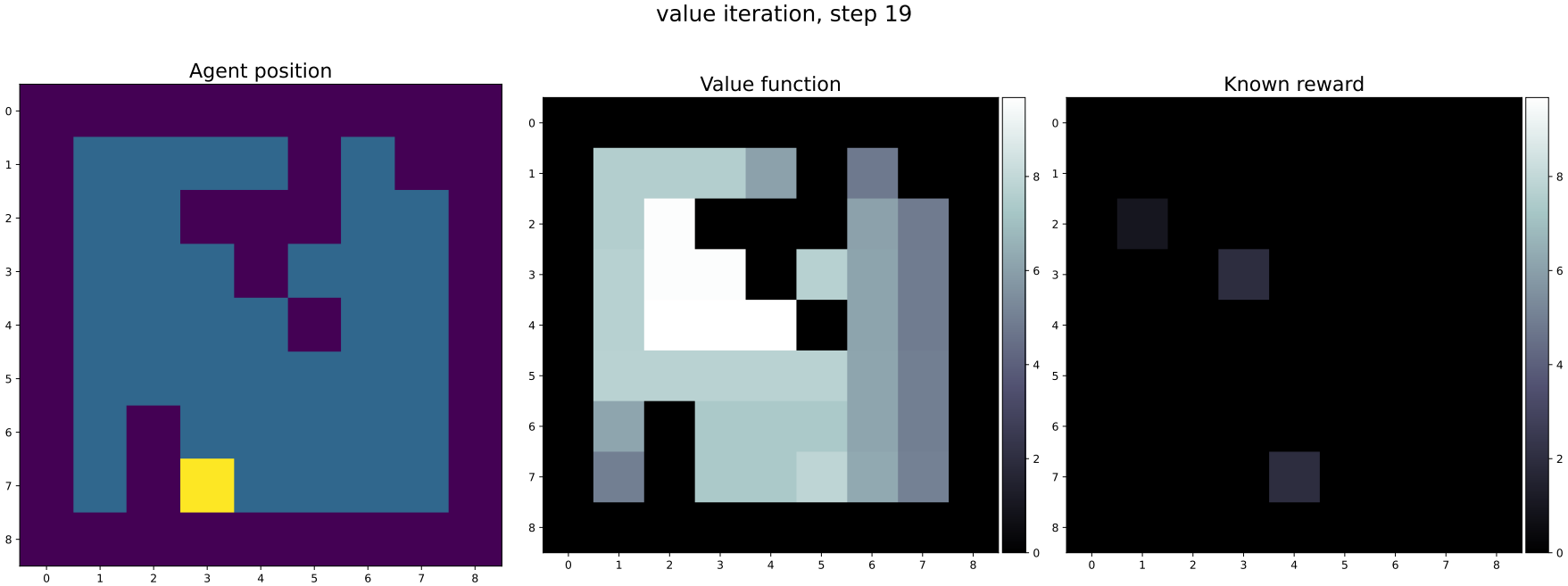


Figure – Iteration 19

Value iteration

After a number of iterations, we observe the convergence of the value function.

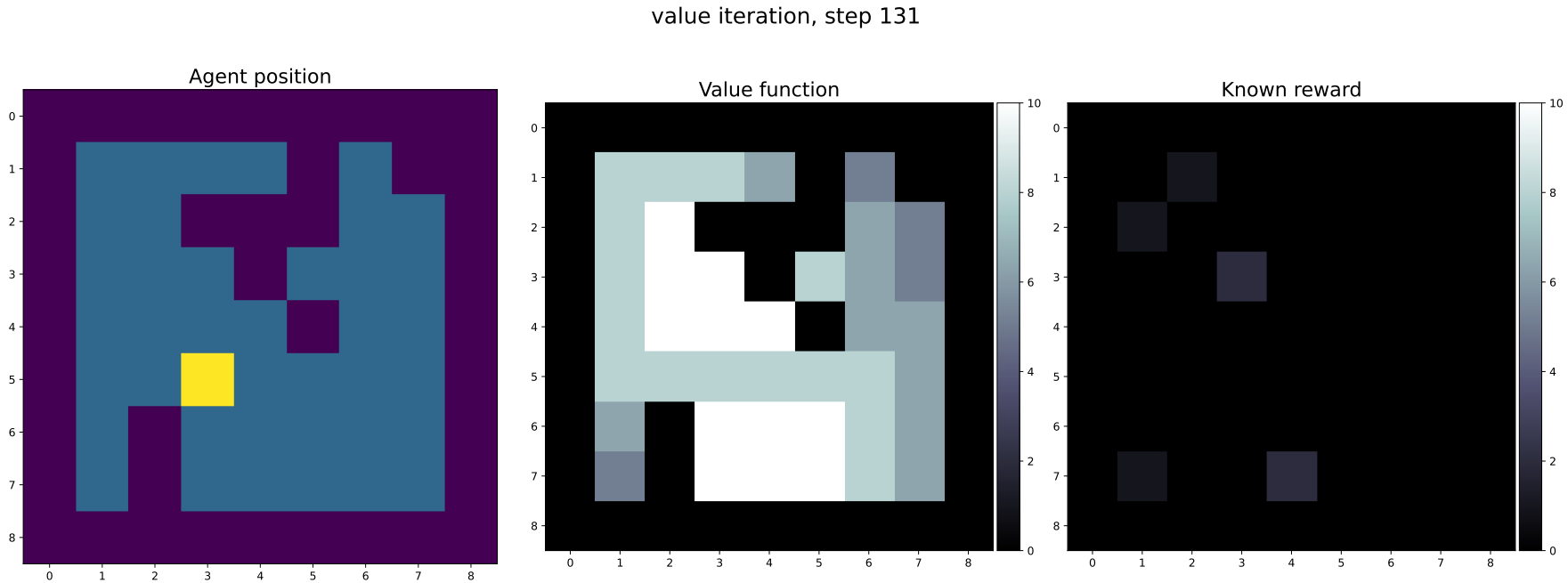


Figure – Iteration 131

Optimal policy

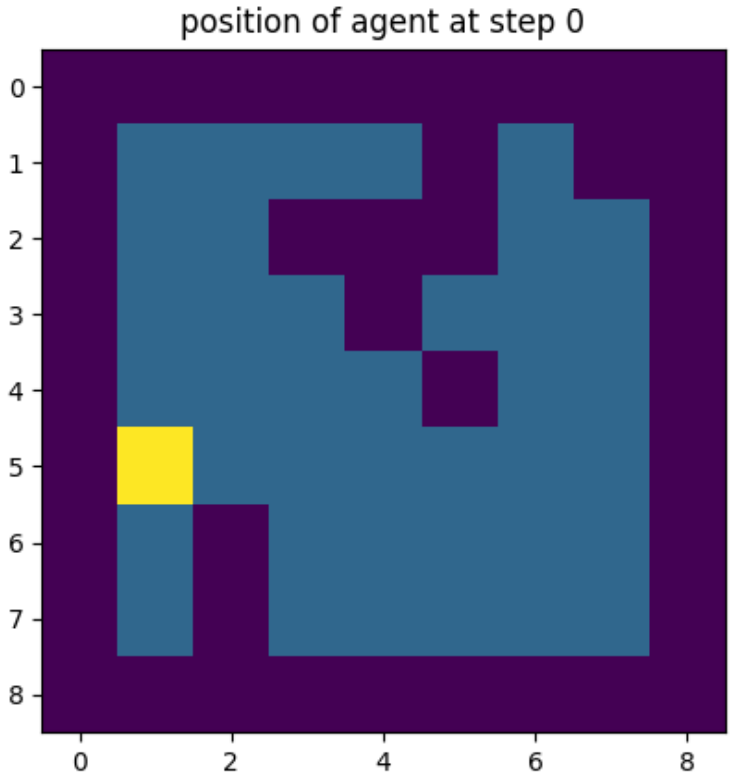


Figure – After learning the optimal policy, the agent can go to the reward.

Multiple paradigms

- Reinforcement learning has many variants.
- In the ones we studied, a model of the consequence of our actions was known.
- This is not always the case.

Temporal difference learning

- In temporal difference learning, the agent does not know a **model** of its world.
- But it can still learn the value function with the **TD (temporal difference) updates**

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] \quad (8)$$

Monte Carlo methods

Monte Carlo methods can be used in Reinforcement Learning to estimate some expected values (such as the expected reward in a given state).

Actor critic methods

- Sometimes you can use **two** policies
  - the **behavior policy** provides actions and guarantees exploration
  - the **target policy** is the optimal policy learned in parallel by the agent, that would be used in exploitation mode.

Tabular case and continous case

- We studied **finite** (and thus discrete situations).
- However, RL can also be applied to continuous state / discrete action spaces (DQN)
- And even to continous state / continous action spaces (DDPG) [Bengio, 2009] .

Regression

- $\mathcal{X} = \mathbb{R}^d$
- $\mathcal{Y} = \mathbb{R}$.

Until now we used the squared error or the mean squared error (MSE) :

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_{pred,i} - y_{label,i})^2 \tag{9}$$

Without normalisation, it is also called **residual sum of squares** (RSS)

$$RSS = \sum_{i=1}^{n}(y_{pred,i} - y_{label,i})^2 \tag{10}$$

Coefficient of determination

Also called $R2$. $R2 \leq 1$. We introduce the Total sum of squares (TSS)

$$TSS = \sum_{i=1}^{n} (y_{label,i} - \bar{y})^2 \tag{11}$$

where $\bar{y}$ is the mean of the labels :

$$\bar{y} = \frac{1}{n} \sum_{i=1}^{n} y_{label,i} \tag{12}$$

Then

$$R2 = 1 - \frac{RSS}{TSS} \tag{13}$$

Coefficient of determination

$$TSS = \sum_{i=1}^{n} (y_{label,i} - \bar{y})^2 \tag{14}$$

$$RSS = \sum_{i=1}^{n} (y_{pred,i} - y_{label,i})^2 \tag{15}$$

Finally we define the Explained sum of squares ESS :

$$ESS = \sum_{i=1}^{n} (y_{pred,i} - \bar{y})^2 \tag{16}$$

Then if the predicitons are linear, then

$$TSS = ESS + RSS \tag{17}$$

Scikit metrics

https://scikit-learn.org/stable/modules/model_evaluation.html

Binary classification

We now review some metrics for binary classification problems.

Accuracy

Most common scoring : accuracy.

$$\frac{\text{number of correct predictions}}{\text{number of samples}} \tag{18}$$

Precision and recall

Precision : "Quand tu dis que c'est positif, c'est positif".

$$\frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}} \tag{19}$$

Recall / sensitivity (rappel) : "Quand c'est positif, tu dis que c'est positif".

$$\frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \tag{20}$$

See also : specificity and 1-specificity.
`https://scikit-learn.org/stable/auto_examples/model_selection/plot_precision_recall.html`

F score

Also called $F1$. It quantifies the tradeoff between precision and recall.

$$F1 = 2 \times \frac{\text{sensitivity} \times \text{precision}}{\text{sensitivity} + \text{precision}} \tag{21}$$

$$F \in [0, 1] \tag{22}$$

`https://en.wikipedia.org/wiki/F-score`

Binary classification

Standard classifiers such as logistic regression and support vector machines are **binary**.
However, sometimes $|\mathcal{Y}| = p > 2$. In these situations, several binary classifiers are aggregated in order to perform the classification.

- one-vs-rest scheme : learns a binary classifier per class. At prediction time, select the class with highest score (there is often some form of confidence score associated with a binary classifier)
- one-vs-one scheme : leanrs as many binary classifiers as there are pairs of classes.

Number of classifiers

We assume $|\mathcal{Y}| = p$.

Exercice 2 : How many classifiers need to be built :

- with the one-vs-rest scheme ?
- with the one-vs-one scheme ?

Number of classifiers

- one-vs-rest is the standard approach.
- one-vs-one need to build a number of classifiers that is quadratic in p, ($\mathcal{O}(p^2)$).
- However one-vs-one might still be useful since less samples are used by each binary classifiers, since we only need the samples from the two selected classes. If the complexity of the classifier scales badly with the umber of samples n (like for kernel methods), one-vs-one might be faster.

Scikit

Scikit has a builtin implementation of both schemes.
`https://scikit-learn.org/stable/modules/generated/sklearn.multiclass.OneVsRestClassifier.html`

Softmax

It is also possible to directly learn p real outputs and predict the maximum. But during training, we need something that we can differentiate.
The softmax approach applies the softmax function to the p outputs, and we train the model to have a softmaxed output close to a discrete disctribution.
`https://fr.wikipedia.org/wiki/Fonction_softmax`

Confusion matrix

https://en.wikipedia.org/wiki/Confusion_matrix
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html

Classification report

It is also possible to define precision, recall (and thus F1) for each class. In scikit, classification report prints these quantities for each class

`https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html`

Multi-class vs multi-label

Don't mix multi-class problems and multi-label problems.

- multi-class : several output classes are possible
- mutli-label : we have to make several predictions for each input

A problem can be both multi-class and multi-label.

Learning curves

https://scikit-learn.org/stable/auto_examples/model_selection/plot_learning_curve.html

Many model selection methods exist :

https://scikit-learn.org/stable/model_selection.html

Classification threshold

https://scikit-learn.org/stable/modules/classification_threshold.html

References I


Bengio, Y. (2009).
CONTINUOUS CONTROL WITH DEEP REINFORCEMENT LEARNING.
Foundations and Trends® in Machine Learning.


Mnih, V., Silver, D., and Riedmiller, M. (2013).
Deep Q Network (Google).
Arxiv.


Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., and Hassabis, D. (2016).

References II

Mastering the Game of Go with Deep Neural Networks and Tree Search.
Nature, 529(7587) :484–489.